14

DEEP DIVE 14 · CTRL+Z వెనక ఏమి ఉంది

Design a Text Editor with Undo

> Google Docs · VS Code · Notion · Figma – ప్రతి ఒక్కరూ రోజూ వాడే feature, మరియు దాన్ని సరిగ్గా చేయడం ఆశ్చర్యకరంగా కష్టం.

మూడు విరుపులు. మొదటిది ఒక్క keystroke కి **20 లక్షల అక్షరాలని కాపీ** చేస్తుంది. రెండోది దాన్ని సరిచేసి **వేరే విధంగా నెమ్మదవుతుంది**. మూడోది undo history ని **4.8 GB** చేసి, ఆపై **మీరు ఇప్పుడే రాసిన వాక్యాన్ని మింగేస్తుంది.**

తెలుగు + ENGLISH

YASWANTH · DEEP DIVE 14

ఈ doc ఎలా చదవాలి

పక్కన editor తెరిచి కలిసి రాయండి. ఇక్కడి **ప్రతి output, ప్రతి సంఖ్య నిజంగా `node` lo run చేసినదే.**

ఈ DOC LO ఒక కొత్త పద్ధతి – FUZZ TESTING

ఈ problem lo "సరైనది" అంటే చాలా సూక్ష్మమైనది: `insert`, `delete`, `undo`, `redo` లని **ఏ క్రమంలో కలిపినా** సరైన ఫలితం రావాలి. కొన్ని tests రాసి "పనిచేస్తోంది" అనడం సరిపోదు.

కాబట్టి ఇక్కడ రెండు చోట్ల **fuzz test** వాడాను – వేలాది యాదృచ్ఛిక operations నడిపి, ప్రతి అడుగులోనూ ఒక సాదా reference తో పోల్చడం. §5 lo 36,000 operations, §13 lo 30,000.

ఇది interview lo చెప్పదగిన విషయం: **"I'd fuzz undo/redo against a naive reference implementation."**

విషయ సూచిక (Table of Contents)

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. అడిగింది ఏమిటి – మరియు ఇక్కడ రెండు వేర్వేరు problems ఉన్నాయి
2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

Part 2 – మొదటి విరుపు: document ఒక string

3. Step – `slice` + `+` · అతి సహజమైన రూపం
4. **మొదటి విరుపు** – ఒక అక్షరం type చేయడానికి 20 లక్షల కాపీలు
5. Step – Piece table · అక్షరాలని ఎప్పుడూ కదపకపోవడం

Part 3 – రెండో విరుపు: ముక్కల పేలుడు

6. Step – ఎన్ని pieces తయారవుతున్నాయి?
7. **రెండో విరుపు** – 50,000 edits కి 1,162 ms
8. Step – వరుసగా type చేస్తే ఒకే piece

Part 4 – మూడో విరుపు: Undo

9. Step – ప్రతి మార్పుకీ ఒక పూర్తి నకలు
10. **మూడో విరుపు** – 4.8 GB · మరియు redo మీ వాక్యాన్ని మింగేసింది
11. Step – మార్పుని ఒక వస్తువుగా చేయడం

Part 5 – పూర్తి system

12. Step – 11 keystrokes, ఒకే Ctrl+Z
13. మొత్తం code ఒకే చోట · నడిపి చూద్దాం

Part 6 – Interview lo

14. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి
15. నోటితో చెప్పాల్సిన English script
16. Follow-ups – collaboration, పెద్ద files, search
17. ఏమి నేర్చుకున్నాం

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

---

1. అడిగింది ఏమిటి – మరియు ఇక్కడ రెండు వేర్వేరు problems ఉన్నాయి

> *"Design a text editor supporting insert, delete, undo and redo."*

ఇది ఒక ప్రశ్నలా అనిపిస్తుంది. నిజానికి ఇవి **రెండు**, మరియు అవి ఒకదానితో ఒకటి ముడిపడి ఉన్నాయి:

రెండు PROBLEMS · ఒకటి రెండోదాన్ని నిర్ణయిస్తుంది

Text ని ఎలా నిల్వ చేస్తారో, undo ఎలా ఉంటుందో నిర్ణయిస్తుంది

1 · Text ని ఎలా నిల్వ చేయాలి?

ఒక string అయితే – ప్రతి keystroke కి
మొత్తం document ని **తిరిగి కట్టాలి**
2 MB file lo అది 20 లక్షల కాపీలు
→ **§4 · 314 ms**

2 · Undo ని ఎలా నిల్వ చేయాలి?

ప్రతి స్థితిని దాచితే – memory పేలుతుంది
మార్పుని మాత్రమే దాస్తే – ఆ మార్పుని
వెనక్కి తిప్పడం ఎలా?
→ §10 · **4.8 GB**

మరియు ఒక మూడో విషయం – ఎవరూ ఆలోచించనిది

నేను ఒక వాక్యం రాశాను, **Ctrl+Z** నొక్కాను, ఆపై **వేరే వాక్యం** రాశాను.
ఇప్పుడు **Ctrl+Y** నొక్కితే ఏమి జరగాలి?

సహజమైన అమలు – నేను ఇప్పుడే రాసిన వాక్యాన్ని **తుడిచేసి**, నేను వద్దన్నదాన్ని తెస్తుంది.

ఇది ఒక UI చిన్న లోపం కాదు – ఇది **డేటా నష్టం**. (§10)

---

2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

ప్రశ్న	జవాబు నా design ని ఎలా మారుస్తుంది
Document ఎంత పెద్దది? 1 KB నా 100 MB నా?	§4 – ఇదే మొదటి విరుపుని నిర్ణయిస్తుంది

ప్రశ్న	జవాబు నా design ని ఎలా మారుస్తుంది
Undo ఎంత వెనక్కి? 100 అడుగులా, అపరిమితమా?	అపరిమితం అంటే §10 – memory ఒక design constraint
ప్రతి keystroke ఒక undo అడుగా?	కాదు అంటే **coalescing** (§12) – ఇది UX నిర్ణయం
Undo తర్వాత కొత్త మార్పు చేస్తే redo ఏమవుతుంది?	**ఇదే §10 విరుపు** – మీరే అడగండి
Cursor / selection ని కూడా undo పునరుద్ధరించాలా?	అవును – command lo cursor స్థానం కూడా ఉండాలి
ఒకే సమయంలో పలువురు ఎడిట్ చేస్తారా?	అవును అంటే ఇది **పూర్తిగా వేరే problem** (§16)
Find & replace, syntax highlighting?	సాధారణంగా వదిలేయమంటారు – అడగండి

నాలుగో ప్రశ్న INTERVIEW ని మలుపు తిప్పుతుంది

"నేను ఒక వాక్యం రాసి, undo చేసి, ఆపై వేరే వాక్యం రాశాను. ఇప్పుడు redo నొక్కితే ఏమి జరగాలి?"

సరైన జవాబు: **ఏమీ జరగకూడదు.** నేను కొత్తగా రాసిన క్షణంలోనే ఆ పాత redo **చెల్లనిదైపోయింది** – అది వేరే చరిత్రకి చెందినది.

కానీ మీరు undo మరియు redo ని రెండు stacks గా రాసి ఆగిపోతే – redo **పనిచేస్తుంది**, మరియు అది **నేను ఇప్పుడే రాసిన దాన్ని తుడిచేస్తుంది**.

§10 lo దాన్ని నడిపి చూద్దాం. అది చూస్తే మీరు ఇక ఎప్పుడూ మర్చిపోరు.

Part 2 – మొదటి విరుపు: document ఒక string

---

3. Step – `slice` + `+` · అతి సహజమైన రూపం

```
class Editor {
  constructor(text = '') { this.text = text; }
  insert(at, s) { this.text = this.text.slice(0, at) + s + this.text.slice(at); }
  delete(at, n) { this.text = this.text.slice(0, at) + this.text.slice(at + n); }
  get length() { return this.text.length; }
}
```

మూడు పంక్తులు, మరియు **సరిగ్గా పనిచేస్తాయి**:

```
"Hello, world!"
" world!"
```

మరి సమస్య ఏమిటి? – ఆ `+` ని చూడండి. JavaScript lo (మరియు Java, Python, C# lo కూడా) **strings మార్చలేనివి** (immutable). అంటే ఆ `+` ఒక కొత్త string **తయారు చేస్తోంది** – పాత అక్షరాలన్నిటినీ కొత్త చోటికి **కాపీ చేస్తోంది**.

---

4. మొదటి విరుపు – ఒక అక్షరం type చేయడానికి 20 లక్షల కాపీలు

ఒక document మధ్యలో 2,000 అక్షరాలు type చేసి, **ఎన్ని అక్షరాలు కాపీ అయ్యాయో** లెక్కిద్దాం:

```
ఒక document మధ్యలో 2,000 అక్షరాలు type చేస్తే

   document పరిమాణం |  సమయం   |  కాపీ అయిన అక్షరాలు
   -----------------+---------+---------------------
             10,000 |    2 ms |         2,19,99,000
           2,00,000 |   41 ms |        40,19,99,000
          20,00,000 |  290 ms |      4,00,19,99,000
```

సమయాలు runs మధ్య ఊగుతాయి – 2 ms, 41–61 ms, 290–317 ms. కానీ **కుడి నిలువు వరుస ప్రతిసారీ అక్షరాలా ఒకటే**, ఎందుకంటే అది గడియారం కాదు, **లెక్క**. Deep Dive 13 §7 lo చెప్పిన అదే విషయం: *గడియారం ఊగితే, పనిని లెక్కించండి.*

2,000 అక్షరాలు చేర్చడానికి చేసిన పని

Document పెరిగితే, ఒక్కో keystroke ఖరీదవుతుంది – పనేమీ మారకపోయినా

2 MB file lo ఒక అక్షరం type చేస్తే – **20 లక్షల అక్షరాలు** కదులుతాయి.
నిష్పత్తి: **1 కి 20,00,000**. మరియు file పెద్దదైతే అది మరింత పెరుగుతుంది.
అందుకే పెద్ద files lo Notepad type చేస్తుంటే ఆగిపోతుంది.

మొదటి విరుపు – ఖర్చు మార్పు మీద కాదు, DOCUMENT మీద ఆధారపడుతోంది

నేను **ఒక అక్షరం** చేర్చాను. Document 2 MB అయితే system **20 లక్షల అక్షరాలని** కదిలించింది.

మౌలిక తప్పు: "Document" అనేది **ఒకే అవిభాజ్య వస్తువు** అని అనుకున్నాం. అందుకే ఏ చిన్న మార్పుకైనా మొత్తాన్నీ తిరిగి కట్టాలి.

కానీ నిజానికి ఒక document అంటే – **ముక్కల వరుస**. "ఈ 5 లక్షల అక్షరాలు, ఆపై ఈ కొత్త అక్షరం, ఆపై మిగతా 15 లక్షలు." ఆ ముక్కలని **కదిలించాల్సిన అవసరం లేదు** – వాటి వరుసని మాత్రమే మార్చాలి.

---

5. Step – Piece table · అక్షరాలని ఎప్పుడూ కదపకపోవడం

ఆ ఆలోచనకి ఒక పేరు ఉంది: **piece table**. VS Code దీన్నే వాడుతుంది.

PIECE TABLE · రెండు BUFFERS, ఒక ముక్కల జాబితా

"Hello world" lo 5వ స్థానంలో ',' చేర్చితే

ఒక్క అక్షరం కూడా కాపీ కాలేదు. "Hello world" **ఎక్కడ ఉందో అక్కడే** ఉంది.
మారినది **ముక్కల జాబితా** మాత్రమే – అది మూడు చిన్న వస్తువులు.

```
class PieceTable {
  constructor(original = '') {
    this.orig = original;              // ఎప్పటికీ మారదు
    this.add  = '';                    // చివరికి మాత్రమే చేరుస్తాం
    this.pieces = original.length
      ? [{ buf: 'orig', start: 0, len: original.length }] : [];
  }
  #text(p) { return (p.buf === 'orig' ? this.orig : this.add)
                   .substr(p.start, p.len); }
```

```
  toString() { return this.pieces.map(p => this.#text(p)).join(''); }
  get length() { return this.pieces.reduce((s, p) => s + p.len, 0); }

  // స్థానం 'at' ఏ piece lo, ఎక్కడ పడుతుందో
  #locate(at) {
    let off = 0;
    for (let i = 0; i < this.pieces.length; i++) {
      if (at <= off + this.pieces[i].len) return { i, inner: at - off };
      off += this.pieces[i].len;
    }
    return { i: this.pieces.length, inner: 0 };
  }

  insert(at, s) {
    if (!s) return;
    const piece = { buf: 'add', start: this.add.length, len: s.length };
    this.add += s;                                  // append మాత్రమే
    const { i, inner } = this.#locate(at);
    if (i >= this.pieces.length) { this.pieces.push(piece); return; }
    const p = this.pieces[i];
    if (inner === 0) this.pieces.splice(i, 0, piece);
    else if (inner === p.len) this.pieces.splice(i + 1, 0, piece);
    else this.pieces.splice(i, 1,                   // piece ని రెండుగా చీల్చడం
      { buf: p.buf, start: p.start, len: inner },
      piece,
      { buf: p.buf, start: p.start + inner, len: p.len - inner });
  }
}
```

`delete` కూడా అదే ఆలోచన – తాకిన pieces ని **కత్తిరించడం లేదా తొలగించడం**, అక్షరాలని కాపీ చేయకుండా. (పూర్తి code §13 lo.)

ముందు: ఇది నిజంగా సరైనదేనా?

ఇక్కడ చాలా సూక్ష్మమైన సరిహద్దులు ఉన్నాయి – piece మొదట్లో చేర్చడం, చివర్లో చేర్చడం, మధ్యలో చీల్చడం, పలు pieces మీదుగా delete చేయడం. కొన్ని tests రాసి "పనిచేస్తోంది" అనడం **చాలదు**.

కాబట్టి **fuzz test**: వేలాది యాదృచ్ఛిక operations నడిపి, ప్రతి అడుగులోనూ సాదా string తో పోల్చడం.

```
for (let trial = 0; trial < 300; trial++) {
  let ref = 'start-' + trial;
  const pt = new PieceTable(ref);
  for (let op = 0; op < 120; op++) {
    // యాదృచ్ఛికంగా insert లేదా delete – రెండింటి మీదా
    ...
    if (pt.toString() !== ref) { /* విఫలం */ }
  }
}
```

```
300 trials × 120 operations · piece table = సాదా string? అవును ✓
```

36,000 operations, ఒక్క తేడా లేదు. ఇప్పుడు వేగం కొలవొచ్చు.

```
ఒక document మధ్యలో 2,000 అక్షరాలు type చేస్తే

   document పరిమాణం |  string  →  piece table
   -----------------+-----------------------
             10,000 |    2 ms  →  3 ms    ✓
           2,00,000 |   53 ms  →  3 ms    ✓
          20,00,000 |  330 ms  →  3 ms    ✓
```

Piece table మూడు సందర్భాల్లోనూ 3 ms. Document పరిమాణంతో సంబంధం లేదు – ఎందుకంటే **అక్షరాలు ఎప్పుడూ కదలవు**.

ఇక్కడా runs మధ్య ఊగిసలాట ఉంది – string వరుస 1-2 / 43-61 / 290-330 ms, piece table వరుస 2-3 ms. ఒక్కో సంఖ్యని కాదు, **రెండు నిలువు వరుసల ఆకారాన్ని** చూడండి: ఒకటి పెరుగుతోంది, రెండోది పెరగట్లేదు. **వేగం కొలిచే ముందు సరైనత పోల్చడం** – ఇది Deep Dive 11 §8 మరియు 13 §5 lo కూడా చేశాం, మరియు రెండుసార్లూ అది ఒక bug పట్టుకుంది. ఇక్కడ పట్టుకోలేదు, కానీ ఆ 36,000 operations లేకుండా నేను ఈ 3 ms ని నమ్మేవాడిని కాదు.

Part 3 – రెండో విరుపు: ముక్కల పేలుడు

6. Step – ఎన్ని pieces తయారవుతున్నాయి?

`#locate` ని మళ్ళీ చూడండి:

```
#locate(at) {
  let off = 0;
  for (let i = 0; i < this.pieces.length; i++) {      // ← pieces అన్నిటి మీదా
    if (at <= off + this.pieces[i].len) return { i, inner: at - off };
    off += this.pieces[i].len;
  }
  ...
}
```

ఇది **pieces జాబితా అంతటినీ** నడుస్తుంది. §5 lo అది సరిపోయింది, ఎందుకంటే pieces కొన్ని వందలే ఉన్నాయి.

కానీ **ప్రతి insert ఒక piece ని మూడుగా చీలుస్తుంది**. అంటే pieces సంఖ్య edits తో పాటు పెరుగుతుంది. ఎంత పెరుగుతుంది?

---

7. రెండో విరుపు – 50,000 edits కి 1,162 ms

2 MB document, మధ్యలో వరుసగా type చేస్తూ:

```
   2000 edits →     3 ms ·   2002 pieces
  10000 edits →    44 ms ·  10002 pieces
  50000 edits →  1162 ms ·  50002 pieces
```

Pieces సంఖ్య **ఖచ్చితంగా నిర్ణయాత్మకమైనది** – 2002, 10002, 50002, ప్రతి run lo. సమయాలు ఊగుతాయి (50,000 వరుస 1,162–1,513 ms), కానీ ఆ **O(n²) ఆకారం** మారదు: edits 5 రెట్లు → సమయం 26–34 రెట్లు.

రెండో విరుపు – ఒక సమస్యని రెండోదానితో మార్చుకున్నాం

§4 lo ఖర్చు **document పరిమాణం** మీద ఆధారపడేది. ఇప్పుడు అది **edits సంఖ్య** మీద ఆధారపడుతోంది. రెండూ పెరుగుతూనే ఉంటాయి.

మరియు ఇది **ఘోరంగా** పెరుగుతోంది: pieces n అయితే, ప్రతి `#locate` O(n), మరియు n edits చేస్తే మొత్తం **$O(n^2)$**. 5 రెట్లు ఎక్కువ edits → **26 రెట్లు** ఎక్కువ సమయం.

50,000 edits అంటే ఏమిటి? – ఒక రోజు పని. ఒక programmer సాధారణ session lo అంత type చేస్తాడు. అంటే **మధ్యాహ్నానికి editor ఆగిపోతుంది.**

కానీ ఒక్క క్షణం – pieces నిజంగా అన్ని కావాలా?

ఆ 50,002 pieces ని చూడండి. నేను చేసినది **వరుసగా type చేయడం** – a, ఆపై దాని పక్కన a, ఆపై దాని పక్కన a. అంటే:

```
piece 1: add[0..1]   piece 2: add[1..2]   piece 3: add[2..3]   ...
```

ఇవన్నీ **`add` buffer lo పక్కపక్కనే** ఉన్నాయి, మరియు **document lo కూడా పక్కపక్కనే** ఉన్నాయి. అంటే ఇవి **ఒకే piece** కావచ్చు: `add[0..50000]`.

---

8. Step – వరుసగా type చేస్తే ఒకే piece

`insert` మొదట్లో ఒక తనిఖీ చేరిస్తే చాలు:

```
insert(at, s) {
  if (!s) return;
  const { i, inner } = this.#locate(at);
  const prev = this.pieces[i];
  // వరుసగా type చేస్తుంటే – కొత్త piece వద్దు, ఉన్నదాన్నే పొడిగించు
  if (prev && prev.buf === 'add' && inner === prev.len &&
      prev.start + prev.len === this.add.length) {
    this.add += s; prev.len += s.length; return;
  }
  ...
}
```

మూడు షరతులు, మరియు **మూడూ అవసరం**:

- **`prev.buf === 'add'`** – అది `add` buffer నుంచి రావాలి (`orig` ని పొడిగించలేం, అది మారదు).
- **`inner === prev.len`** – నేను ఆ piece యొక్క **సరిగ్గా చివర్లో** type చేస్తున్నాను.
- **`prev.start + prev.len === this.add.length`** – ఆ piece `add` buffer యొక్క **చివరి దాకా** ఉంది, అంటే కొత్త అక్షరం దాని పక్కనే పడుతుంది.

మూడోది మర్చిపోవడం చాలా సులభం, మరియు అది **నిశ్శబ్ద డేటా పాడు** – user వేరే చోటికి వెళ్ళి type చేస్తే, ఆ పాత piece తప్పుగా పొడిగించబడుతుంది. **Fuzz test ఇలాంటిదే పట్టుకుంటుంది.**

```
300 trials × 120 operations · piece table = సాదా string? అవును ✓

    2000 edits →     0 ms ·      3 pieces
   10000 edits →     1 ms ·      3 pieces
   50000 edits →     2 ms ·      3 pieces
  200000 edits →     8 ms ·      3 pieces
```

50,002 pieces → 3. 1,162 ms → 2 ms. మరియు 2 లక్షల edits కి కూడా **3 pieces, 8 ms**.

ఇది అన్ని నమూనాలనీ పరిష్కరించదు. User document అంతటా అటూ ఇటూ దూకుతూ edit చేస్తే, pieces మళ్ళీ పెరుగుతాయి. నిజమైన editors (VS Code) అందుకే pieces ని ఒక **array lo కాదు, ఒక balanced tree lo** ఉంచుతాయి – అప్పుడు `#locate` O(log n).

Interview lo ఇది చెప్పడం ముఖ్యం: "Coalescing handles sequential typing, which is the dominant pattern. For random-access editing you'd want the pieces in a balanced tree so locate is logarithmic." – ఇది మీరు హద్దులు తెలిసి ఎంచుకుంటున్నారని చూపిస్తుంది.

Part 4 – మూడో విరుపు: Undo

9. Step – ప్రతి మార్పుకీ ఒక పూర్తి నకలు

ఇప్పుడు undo. అతి సహజమైన ఆలోచన: **ప్రతి మార్పుకీ ముందు, document ని దాచిపెట్టడం.**

```
class SnapshotEditor {
  constructor(text = '') { this.text = text; this.history = [text]; }
  #save() { this.history.push(this.text); }
  insert(at, s) { this.text = this.text.slice(0,at)+s+this.text.slice(at);
                  this.#save(); }
  undo() {
    if (this.history.length > 1) {
      this.history.pop();
      this.text = this.history[this.history.length - 1];
    }
  }
}
```

ఇది **సరిగ్గా పనిచేస్తుంది**, మరియు ఒకే ఒక పంక్తి undo. రాయడానికి ఇంతకంటే సులభమైనది లేదు.

10. మూడో విరుపు – 4.8 GB · మరియు redo మీ వాక్యాన్ని మింగేసింది

మొదటి సగం – memory

```
ఒక document lo type చేస్తూ – undo history ఎంత పెరుగుతుంది

   document |  50 keystrokes |  500 |  5,000
   ---------+----------------+-------+--------
       1 KB |          0.1 MB | 1.2 MB | 33.4 MB
      50 KB |          4.9 MB | 48.0 MB | 500.8 MB
     500 KB |         48.6 MB | 478.0 MB | 4793.2 MB
```

ఒక సాధారణ పని SESSION = 4.8 GB

500 KB అంటే ఒక పెద్ద source file, లేదా ఒక అధ్యాయం. 5,000 keystrokes అంటే **ఒక గంట పని**.

అంటే ఒక గంట తర్వాత editor **4.8 GB** RAM వాడుతోంది – **ఒక్క file కోసం**.

50 KB (ఒక మామూలు source file) కూడా **500 MB**. ఇది ఒక tab. పది tabs తెరిస్తే?

రెండో సగం – మరియు ఇది ఇంకా ఘోరం

Redo చేర్చుదాం. సహజమైన రూపం – రెండు stacks:

```
undo() { this.redoStack.push(this.text); this.text = this.undoStack.pop(); }
redo() { this.undoStack.push(this.text); this.text = this.redoStack.pop(); }
```

ఇప్పుడు ఒక సాధారణ పని:

```
"Dear Ravi" అని రాశాను              "Dear Ravi"
ఒక వాక్యం చేర్చాను               "Dear Ravi, thanks for the help"
undo – ఆ వాక్యం వద్దు              "Dear Ravi"
బదులుగా ఇది రాశాను                "Dear Ravi, see you Monday"

ఇప్పుడు redo నొక్కితే →          "Dear Ravi, thanks for the help"
```

మూడో విరుపు – "SEE YOU MONDAY" ఎక్కడికి పోయింది?

నేను ఇప్పుడే రాసిన వాక్యం **పూర్తిగా మాయమైంది**, మరియు నేను స్పష్టంగా **వద్దన్న** వాక్యం తిరిగి వచ్చింది.

ఇంకా ఘోరం: ఆ కోల్పోయిన text ని **undo చేసి తిరిగి తేలేను** కూడా. అది చరిత్ర నుంచే తొలగిపోయింది.

మౌలిక తప్పు: Redo stack అనేది "ఒకవేళ user మనసు మార్చుకుంటే" అని దాచిన **ఒక ప్రత్యామ్నాయ భవిష్యత్తు**. User కొత్తగా ఏదైనా రాసిన క్షణం – **ఆ భవిష్యత్తు చనిపోయింది**. దాన్ని ఉంచడం అంటే, రెండు వేర్వేరు చరిత్రలని కలపడం.

పరిష్కారం ఒక్క పంక్తి: కొత్త మార్పు వచ్చినప్పుడు `redoStack.length = 0`.

చరిత్ర ఒక రేఖ · ఒక కొమ్మ కాదు

Undo తర్వాత కొత్తగా రాస్తే – పాత శాఖ చనిపోతుంది

D కి వచ్చాక, C **ఇక చరిత్రలో భాగం కాదు**. దాన్ని redo lo ఉంచడం అంటే –
user ని **ఒక శాఖ నుంచి మరో శాఖకి** దూకించడం, అతనికి చెప్పకుండా.
అందుకే: కొత్త మార్పు → redo stack ఖాళీ.

---

11. Step – మార్పుని ఒక వస్తువుగా చేయడం

Memory సమస్యకి పరిష్కారం: **స్థితిని దాచొద్దు – మార్పుని దాచు.**

మరియు ఆ మార్పు తనని తాను **వెనక్కి తిప్పుకోగలగాలి**. అంటే ప్రతి మార్పూ ఒక వస్తువు, దానికి `do` మరియు `undo` ఉంటాయి:

```
class InsertCommand {
  constructor(at, text) { this.at = at; this.text = text; }
  do(doc)   { doc.insert(this.at, this.text); }
  undo(doc) { doc.delete(this.at, this.text.length); }
}

class DeleteCommand {
  constructor(at, n) { this.at = at; this.n = n; this.removed = null; }
  do(doc)   { this.removed = doc.delete(this.at, this.n); }   // ← ఇక్కడే పట్టుకోవాలి
  undo(doc) { doc.insert(this.at, this.removed); }
}
```

ఆ ఒక్క పంక్తి – `THIS.REMOVED = DOC.DELETE(...)` – అత్యంత ముఖ్యమైనది

`InsertCommand` ని వెనక్కి తిప్పడం సులభం: ఎంత చేర్చామో అంత తీసేయడం. చేర్చిన text మన దగ్గరే ఉంది.

కానీ `DeleteCommand` ని వెనక్కి తిప్పాలంటే – **ఏమి తీసేశామో తెలియాలి**. మరియు అది **తీసేసే క్షణంలోనే** తెలుసుకోవాలి; ఆ తర్వాత అది document lo లేదు.

చాలా మంది `DeleteCommand(at, n)` అని రాసి ఆగిపోతారు – ఆపై `undo` lo "ఏమి తిరిగి పెట్టాలి?" అని కూర్చుంటారు. **జవాబు: అది ఎప్పుడో పోయింది.**

నియమం: ఒక command తనని తాను వెనక్కి తిప్పుకోడానికి కావాల్సినదంతా **అది నడిచే క్షణంలోనే** పట్టుకోవాలి – తర్వాత కాదు.

ఇప్పుడు `History` – మరియు అందులోనే §10 యొక్క రెండో సగానికి పరిష్కారం:

```
class History {
  #undo = []; #redo = [];
  push(cmd) {
    this.#redo.length = 0;          // ← కొత్త మార్పు → redo చెల్లదు
    this.#undo.push(cmd);
  }
  undo(doc) {
    const cmd = this.#undo.pop();
    if (!cmd) return null;
    cmd.undo(doc); this.#redo.push(cmd);
    return cmd;
  }
  redo(doc) {
    const cmd = this.#redo.pop();
    if (!cmd) return null;
    cmd.do(doc); this.#undo.push(cmd);
    return cmd;
  }
}
```

Memory మళ్ళీ కొలుద్దాం

```
5,000 keystrokes తర్వాత undo history పరిమాణం

   document |  snapshots  |  commands  |  తేడా
```

```
---------+-------------+------------+--------
     1 KB |    33.39 MB |   127.0 KB | 269×
    50 KB |   500.78 MB |   127.0 KB | 4,039×
   500 KB |  4793.17 MB |   127.0 KB | 38,662×
```

127 KB – మూడు సందర్భాల్లోనూ ఒకటే

Document 1 KB నా, 500 KB నా – undo history **127 KB**. ఎందుకంటే అది document ని కాదు, **మార్పులని** నిల్వ చేస్తోంది, మరియు మార్పులు రెండు సందర్భాల్లోనూ ఒకటే (5,000 అక్షరాలు).

4.8 GB → 127 KB. 38,662 రెట్లు.

మరియు ఇది ఈ series lo పదేపదే వచ్చిన అదే వాక్యం: ***ఖర్చు ఏమి మారిందో దాని మీద ఆధారపడాలి, మొత్తం ఎంత ఉందో దాని మీద కాదు.*** (Deep Dive 12 §5, 13 §8)

Part 5 – పూర్తి system

12. Step – 11 keystrokes, ఒకే Ctrl+Z

ఒక చివరి సమస్య మిగిలింది, మరియు అది **UX**.

"hello world" అని type చేస్తే అది **11 keystrokes** – అంటే **11 undo commands**. Ctrl+Z నొక్కితే ఒక్క అక్షరం పోతుంది. మొత్తం పదాన్ని తీసేయాలంటే **11 సార్లు**.

ఏ నిజమైన editor అలా ప్రవర్తించదు. అవి **వరుసగా type చేసినదాన్ని ఒకే undo అడుగుగా కలుపుతాయి** – దీన్ని **coalescing** అంటారు.

```
const COALESCE_MS = 1000;
push(cmd, now = Date.now()) {
  this.#redo.length = 0;
  const top = this.#undo[this.#undo.length - 1];
  if (top instanceof InsertCommand && cmd instanceof InsertCommand &&
      now - this.#lastAt < COALESCE_MS &&
      top.at + top.text.length === cmd.at && !cmd.text.includes('\n')) {
    top.text += cmd.text;                     // ఒకే undo అడుగుగా కలపడం
  } else this.#undo.push(cmd);
  this.#lastAt = now;
}
```

నాలుగు షరతులు, మరియు ప్రతిదీ ఒక UX నిర్ణయం:

షరతు	ఎందుకు
రెండూ `InsertCommand`	Type చేయడం మరియు తీసేయడం వేర్వేరు ఆలోచనలు – కలపకూడదు
1 సెకను లోపల	User ఆగి ఆలోచించి మళ్ళీ మొదలుపెడితే, అది ఒక **కొత్త ఆలోచన**
పక్కపక్కనే (`top.at + len === cmd.at`)	User cursor ని జరిపితే అది ఒక కొత్త అడుగు
`\n` **కాదు**	కొత్త పంక్తి అంటే ఒక సహజమైన విరామం

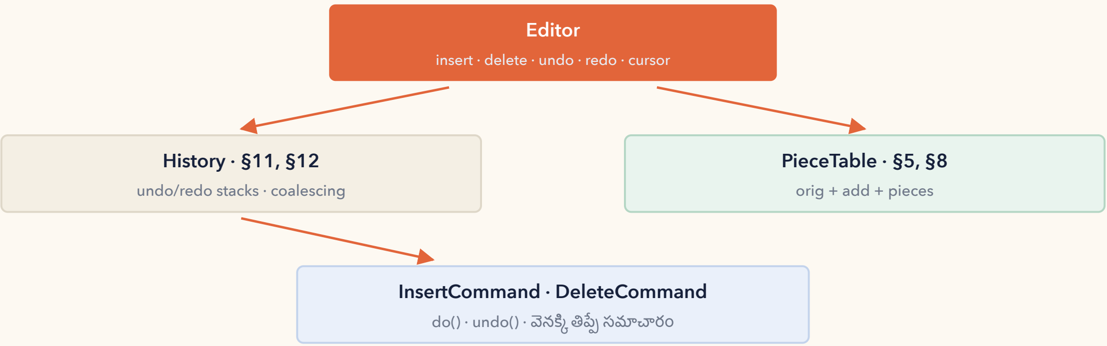
ఆ `now` parameter గమనించండి – నేను `Date.now()` ని **నేరుగా పిలవట్లేదు**, దాన్ని **లోపలికి పంపుతున్నాను**. అందుకే §13 lo coalescing ని **పరీక్షించగలుగుతున్నాను** – "ఇప్పుడు 5 సెకన్లు గడిచాయి" అని చెప్పగలను, నిజంగా ఆగకుండా.

నియమం: సమయం మీద ఆధారపడే ఏ logic ఐనా, సమయాన్ని *తీసుకోవాలి – అడగకూడదు*. Deep Dive 04 §10 lo గడువు తీరడానికి ఇదే చేశాం.

13. మొత్తం code ఒకే చోట · నడిపి చూద్దాం

నిర్మాణం · నాలుగు పొరలు

ప్రతి పొరా ఒక విరుపుకి జవాబు



Commands document ని **మార్చుతాయి**; History commands ని **గుర్తుపెట్టుకుంటుంది**.

```
'use strict';
class PieceTable {
  constructor(original = '') {
    this.orig = original;                  // ఎప్పటికీ మారదు
    this.add  = '';                        // చివరికి మాత్రమే చేరుస్తాం
    this.pieces = original.length
      ? [{ buf: 'orig', start: 0, len: original.length }] : [];
  }
  #buf(name) { return name === 'orig' ? this.orig : this.add; }
  #text(p) { return this.#buf(p.buf).substr(p.start, p.len); }
  get length() { return this.pieces.reduce((s, p) => s + p.len, 0); }
  toString() { return this.pieces.map(p => this.#text(p)).join(''); }

  #locate(at) {
    let off = 0;
    for (let i = 0; i < this.pieces.length; i++) {
      if (at <= off + this.pieces[i].len) return { i, inner: at - off };
      off += this.pieces[i].len;
    }
    return { i: this.pieces.length, inner: 0 };
  }

  insert(at, s) {
    if (!s) return;
    const { i, inner } = this.#locate(at);
    const prev = this.pieces[i];
    // వరుసగా type చేస్తుంటే — కొత్త piece వద్దు, ఉన్నదాన్నే పొడిగించు
    if (prev && prev.buf === 'add' && inner === prev.len &&
        prev.start + prev.len === this.add.length) {
      this.add += s; prev.len += s.length; return;
    }
    const piece = { buf: 'add', start: this.add.length, len: s.length };
    this.add += s;
    if (i >= this.pieces.length) { this.pieces.push(piece); return; }
    const p = this.pieces[i];
    if (inner === 0) this.pieces.splice(i, 0, piece);
    else if (inner === p.len) this.pieces.splice(i + 1, 0, piece);
    else this.pieces.splice(i, 1,                 // piece ని రెండుగా చీల్చడం
```

```
      { buf: p.buf, start: p.start, len: inner },
      piece,
      { buf: p.buf, start: p.start + inner, len: p.len - inner });
  }

  delete(at, n) {
    if (n <= 0) return '';
    let removed = '', remaining = n;
    let { i, inner } = this.#locate(at);
    while (remaining > 0 && i < this.pieces.length) {
      const p = this.pieces[i];
      const take = Math.min(p.len - inner, remaining);
      removed += this.#buf(p.buf).substr(p.start + inner, take);
      const left  = inner > 0
        ? { buf: p.buf, start: p.start, len: inner } : null;
      const right = inner + take < p.len
        ? { buf: p.buf, start: p.start + inner + take,
            len: p.len - inner - take } : null;
      const repl = [left, right].filter(Boolean);
      this.pieces.splice(i, 1, ...repl);
      i += repl.length - (right ? 1 : 0);
      remaining -= take; inner = 0;
    }
    return removed;
  }
}

class InsertCommand {
  constructor(at, text) { this.at = at; this.text = text; }
  do(doc)   { doc.insert(this.at, this.text); }
  undo(doc) { doc.delete(this.at, this.text.length); }
  get cursorAfter() { return this.at + this.text.length; }
  get bytes() { return this.text.length * 2 + 24; }
}
class DeleteCommand {
  constructor(at, n) { this.at = at; this.n = n; this.removed = null; }
  do(doc)   { this.removed = doc.delete(this.at, this.n); }  // ← ఇక్కడే పట్టుకోవాలి
  undo(doc) { doc.insert(this.at, this.removed); }
  get cursorAfter() { return this.at; }
  get bytes() { return (this.removed?.length ?? 0) * 2 + 24; }
}

const COALESCE_MS = 1000;
class History {
  #undo = []; #redo = []; #lastAt = -Infinity;
  push(cmd, now = Date.now()) {
    this.#redo.length = 0;                    // ← కొత్త మార్పు → redo చెల్లదు
    const top = this.#undo[this.#undo.length - 1];
    if (top instanceof InsertCommand && cmd instanceof InsertCommand &&
        now - this.#lastAt < COALESCE_MS &&
        top.at + top.text.length === cmd.at && !cmd.text.includes('\n')) {
      top.text += cmd.text;                   // ఒకే undo అడుగుగా కలపడం
    } else this.#undo.push(cmd);
    this.#lastAt = now;
  }
  breakRun() { this.#lastAt = -Infinity; }     // ఇక్కడి నుంచి కొత్త అడుగు
  undo(doc) {
    const cmd = this.#undo.pop();
    if (!cmd) return null;
```

```
    cmd.undo(doc); this.#redo.push(cmd); this.breakRun();
    return cmd;
  }
  redo(doc) {
    const cmd = this.#redo.pop();
    if (!cmd) return null;
    cmd.do(doc); this.#undo.push(cmd); this.breakRun();
    return cmd;
  }
  get depth() { return { undo: this.#undo.length, redo: this.#redo.length }; }
  get bytes() {
    return [...this.#undo, ...this.#redo].reduce((s, c) => s + c.bytes, 0);
  }
}

class Editor {
  constructor(text = '') {
    this.doc = new PieceTable(text);
    this.history = new History();
    this.cursor = 0;
  }
  get text() { return this.doc.toString(); }
  get length() { return this.doc.length; }

  #run(cmd, now) {
    cmd.do(this.doc);
    this.history.push(cmd, now);
    this.cursor = cmd.cursorAfter;
    return { ok: true, cursor: this.cursor };
  }
  insert(at, s, now = Date.now()) {
    if (at < 0 || at > this.length) return { ok: false, reason: `BAD_POS: ${at}` };
    return this.#run(new InsertCommand(at, s), now);
  }
  delete(at, n, now = Date.now()) {
    if (at < 0 || at + n > this.length)
      return { ok: false, reason: `BAD_RANGE: ${at}+${n}` };
    return this.#run(new DeleteCommand(at, n), now);
  }
  undo() {
    const c = this.history.undo(this.doc);
    if (!c) return { ok: false, reason: 'NOTHING_TO_UNDO' };
    this.cursor = c.at;
    return { ok: true, cursor: this.cursor };
  }
  redo() {
    const c = this.history.redo(this.doc);
    if (!c) return { ok: false, reason: 'NOTHING_TO_REDO' };
    this.cursor = c.cursorAfter;
    return { ok: true, cursor: this.cursor };
  }
}
```

```
--- §10 విరుపు మళ్ళీ ---
  "Dear Ravi"                       "Dear Ravi"
  ఒక వాక్యం                        "Dear Ravi, thanks for the help"
  undo                              "Dear Ravi"
```

```
  బదులుగా ఇది                     "Dear Ravi, see you Monday"
  redo చేస్తే          : { ok: false, reason: 'NOTHING_TO_REDO' }
    → text                          "Dear Ravi, see you Monday"

--- వరుసగా type చేస్తే ఒకే undo అడుగు ---
  11 keystrokes తర్వాత : "hello world" { undo: 1, redo: 0 }
  ఒక undo తర్వాత       : ""

--- delete ని undo చేస్తే తీసేసిన text తిరిగి వస్తుందా ---
  delete(5, 7)         : "Hello!"
  undo                 : "Hello, world!"

--- తిరస్కరణలు ---
  పరిధి దాటిన insert  : { ok: false, reason: 'BAD_POS: 999' }
  ఖాళీ undo           : { ok: false, reason: 'NOTHING_TO_UNDO' }
```

మరియు fuzz test – ఇదే అసలు రుజువు

Undo/redo ని కొన్ని tests తో నమ్మలేం. కాబట్టి: 30,000 **యాదృచ్ఛిక** operations – insert, delete, undo, redo కలిపి – ఒక సాదా reference తో ప్రతి అడుగులోనూ పోల్చడం:

```
// reference: పూర్తి స్థితుల జాబితా + ఒక పాయింటర్
let states = [ed.text], at = 0;
// ... insert/delete → states ని at వద్ద కత్తిరించి కొత్తది చేర్చడం
// ... undo → at--, redo → at++
if (ed.text !== states[at]) { /* విఫలం */ }
```

```
200 trials × 150 operations (30,000 ops) — insert/delete/undo/redo యాదృచ్ఛికంగా కలిపి
editor = reference? అవును ✓
```

ఆ reference `states.slice(0, at + 1)` చేస్తుంది – అంటే **అది కూడా redo ని తొలగిస్తుంది**. ఆ ఒక్క పంక్తే నా `#redo.length = 0` ని ధ్రువీకరిస్తోంది. మరియు `redo()` ఏమీ లేనప్పుడు **విఫలమవుతుందా** అని కూడా పరీక్షిస్తుంది – ఎందుకంటే reference lo `at` చివర్లో ఉంటే redo కి చోటు లేదు.

ఈ output ని పంక్తి పంక్తిగా చదువుదాం

`NOTHING_TO_REDO` – §10 lo ఇక్కడే "see you Monday" మాయమైంది. ఇప్పుడు redo **మర్యాదగా నిరాకరిస్తోంది**, మరియు text చెక్కుచెదరలేదు.

`11 keystrokes తర్వాత { undo: 1 }` – పదకొండు keystrokes, **ఒకే** undo అడుగు. ఒక Ctrl+Z మొత్తం "hello world" ని తీసేసింది.

`delete(5, 7)` → `undo` – "Hello!" నుంచి తిరిగి "Hello, world!". ఆ ", world" అనే **ఏడు అక్షరాలూ** `DeleteCommand.removed` **lo** దాగి ఉన్నాయి.

`BAD_POS: 999` – సరిహద్దు తనిఖీ, మరియు అది **ఒక వస్తువు**, `throw` కాదు. UI దాన్ని విస్మరించొచ్చు.

దశల నుంచి ఇక్కడికి – ఏమి చేరింది

ఎక్కడ నుంచి	ఏమి చేరింది	ఎందుకు
§3	`text` ఒక string	మౌలిక అస్థిపంజరం
§4 (విరుపు)	`PieceTable`	ఒక అక్షరానికి 20 లక్షల కాపీలు
§5	Fuzz test	సరిహద్దులు చాలా, కొన్ని tests చాలవు
§7 (విరుపు)	Coalescing (pieces)	50,000 edits → 50,002 pieces, 1,162 ms
§10 (విరుపు a)	`InsertCommand`, `DeleteCommand`	Snapshots 4.8 GB
§10 (విరుపు b)	`#redo.length = 0`	Redo కొత్త text ని మింగేసింది
§11	`removed` ని `do()` lo పట్టుకోవడం	తీసేసినది తర్వాత తెలియదు
§12	Coalescing (undo అడుగులు)	11 keystrokes = 11 Ctrl+Z
§13	`cursor`, `now` parameter	Cursor ని పునరుద్ధరించడం; సమయాన్ని పరీక్షించడం

Part 6 – Interview lo

14. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి

45 నిమిషాల TIME BUDGET

Undo కి ఎక్కువ సమయం – అదే ప్రశ్న యొక్క పేరు

5 నిమి	**Clarify** – పరిమాణం? undo లోతు? redo తర్వాత?
10 నిమి	String ఎందుకు సరిపోదు · piece table ఆలోచన
15 నిమి – Commands	do/undo · **removed ని పట్టుకోవడం**
8 నిమి – Redo	**శాఖ చనిపోయే క్షణం** · coalescing
7 నిమి	fuzz test · collaboration · scale

ఏమి తప్పక చెప్పాలి

1. **String సరిపోదు, మరియు ఎందుకో** (§4) – "ఒక అక్షరం చేర్చడానికి 20 లక్షలు కాపీ." ఇది 20 సెకన్లు.
2. **Commands, snapshots కాదు** (§11) – మరియు `DeleteCommand` **తీసేసిన text ని** `do()` **lo పట్టుకోవాలి**. ఇదే ఇక్కడి అసలు lesson.
3. **Undo తర్వాత కొత్త మార్పు → redo చనిపోతుంది** (§10) – **మరే లేవనెత్తండి**. చరిత్ర ఒక రేఖ, కొమ్మ కాదు.
4. **Coalescing** (§12) – "11 keystrokes shouldn't be 11 undos." ఇది product ఆలోచనని చూపిస్తుంది.
5. **"I'd fuzz undo/redo against a naive reference"** (§13) – ఈ ఒక్క వాక్యం చాలా విలువైనది.

ఏమి వదిలేయాలి

- `PieceTable.delete` **పూర్తిగా రాయొద్దు** – `insert` రాసి, "delete అదే ఆలోచన, pieces ని కత్తిరించడం" అని చెప్పండి. అది ఐదు నిమిషాలు ఆదా చేస్తుంది.
- **Rope / gap buffer ని పోల్చండి, రాయొద్దు** – "piece table, because it also gives cheap undo and the original buffer never changes."
- **Coalescing షరతులని లోతుగా వెళ్ళొద్దు** – "same direction, close in time, adjacent position."
- **Collaboration** – అడిగితేనే (§16).

15. నోటితో చెప్పాల్సిన English script

"Four questions first. How big can a document get? How deep is undo – bounded or unlimited? Is every keystroke its own undo step? And this one matters most: if I type something, undo it, then type something different – what should redo do?

I'll start with storage, because it constrains undo. The obvious model is one string, and inserting is slice-plus-concat. That's correct but the cost is wrong: strings are immutable, so inserting one character into a two-megabyte document copies two million characters. I measured two thousand keystrokes into documents of ten kilobytes, two hundred kilobytes and two megabytes – two, fifty-three and three hundred thirty milliseconds. The work is proportional to the document, not to the edit.

So I'd use a piece table, which is what VS Code uses. Two buffers: the original, which never changes, and an append-only add buffer. The document is a list of pieces, each pointing into one buffer with an offset and a length. Inserting appends to the add buffer and splits one piece into three. No character ever moves, so the same benchmark is flat – three milliseconds at every size.

There's a catch I'd mention: every insert splits a piece, so the piece list grows with edits, and locating a position is linear in pieces. At fifty thousand edits that was over a second. The fix for the common case is coalescing – if you're typing at the end of the piece you just created, extend it instead of splitting. That took fifty thousand pieces down to three. For genuinely random-access editing you'd want the pieces in a balanced tree so locate is logarithmic; that's what VS Code actually does.

For undo, the tempting approach is snapshotting the document before each change. It's one line and it's correct, but a five-hundred-kilobyte file after five thousand keystrokes – about an hour of work – is four point eight gigabytes of history. Instead each change becomes a command object with do and undo. Insert's undo is a delete of the same length. Delete's undo is an insert of the removed text – and the critical part is that delete has to capture what it removed at the moment it executes, because after that the text is gone. People write DeleteCommand with a position and a length and then discover there's nothing to restore. Same benchmark with commands: a hundred and twenty-seven kilobytes, and the same number at every document size, because history now scales with what changed rather than what exists.

On redo – undo and redo as two stacks looks right and has a data-loss bug. If I type a sentence, undo it, type a different sentence, then press redo, the naive version throws away what I just wrote and restores what I rejected. History is a line, not a tree: the moment you make a new edit, the redo branch is unreachable. So pushing a new command clears the redo stack.

Last, coalescing on the history side: typing eleven characters shouldn't be eleven undos. I'd merge consecutive inserts that are adjacent in position, close in time, and not across a newline. And I'd pass the clock in as a parameter rather than calling Date.now inside, so the time-based part is actually testable.

For confidence I wouldn't rely on hand-written cases – I'd fuzz it: thousands of random insert, delete, undo and redo operations checked against a naive reference that keeps every state in a list with a pointer. That reference also catches the redo-clearing rule for free."

16. Follow-ups – collaboration, పెద్ద files, search

Follow-up	జవాబు	మారే classes
"Undo lo cursor/selection కూడా"	Command lo `cursorBefore` కూడా నిల్వ	`InsertCommand`, `DeleteCommand`
"Undo లోతు 100 కి పరిమితం"	`#undo` ని ఒక bounded queue చేయడం	`History` lo 2 పంక్తులు
"Replace (select చేసి type చేయడం)"	`DeleteCommand` + `InsertCommand` ని ఒక **CompositeCommand** గా	**+1 కొత్తది**
"Find & replace all – ఒకే undo"	అదే `CompositeCommand`	**0 కొత్త concepts**
"పంక్తి సంఖ్యలు, 5 లక్షల పంక్తులు"	Piece lo line-break గణన నిల్వ చేసి, prefix sums	`PieceTable`
"ఒకే file, 100 MB"	Piece table ఇప్పటికే సరిపోతుంది – కానీ `orig` ని **memory-map** చేయాలి	Storage layer
"పలువురు ఒకేసారి edit చేస్తే?"	కింద చూడండి	**పూర్తిగా వేరే problem**

Collaboration – ఇది ఒక పెద్ద తలుపు

> *"ఇద్దరు ఒకే సమయంలో edit చేస్తే, `insert(at, s)` అనే ఆలోచనే విరిగిపోతుంది. నేను స్థానం 10 వద్ద type చేస్తున్నప్పుడు మీరు స్థానం 5 వద్ద ఏదో చేర్చారు – ఇప్పుడు నా '10' తప్పు.*
>
> *దీనికి రెండు పరిష్కారాలు ఉన్నాయి:*
>
> - ***OT** (Operational Transformation) – Google Docs. "ఈ operation ని ఆ operation ప్రకారం సర్దుబాటు చేయి" అనే నియమాలు. శక్తివంతమైనది, కానీ నియమాలు రాయడం కష్టం.*
> - ***CRDT** – ప్రతి అక్షరానికీ ఒక **శాశ్వత గుర్తింపు** ఇవ్వడం, స్థానం కాకుండా. అప్పుడు operations ఏ క్రమంలో వచ్చినా ఫలితం ఒకటే.*
>
> ***మరియు undo మరింత కష్టమవుతుంది:** "నా చివరి మార్పుని వెనక్కి తిప్పు" అంటే – మధ్యలో వేరేవాళ్ళు చేసిన మార్పులని ఉంచుతూ నాది మాత్రమే తీసేయాలి. దీన్ని **selective undo** అంటారు, మరియు ఇది ఒక పరిశోధనా అంశం.*
>
> *Interview lo ఇక్కడికి వెళ్ళాల్సిన అవసరం లేదు – కానీ **"single-user undo is a stack; multi-user undo is selective undo, which is much harder"** అని చెప్పడం మీ హద్దులు తెలుసునని చూపిస్తుంది."*

---

17. ఏమి నేర్చుకున్నాం

ఆలోచన	ఇక్కడ ఎలా కనిపించింది	ఇంకెక్కడ వస్తుంది
ఖర్చు మార్పు మీద ఆధారపడాలి	4.8 GB → 127 KB (§11)	Deep Dive 12 §5, 13 §8, ప్రతి diff/sync system
డేటాని కదపొద్దు – సూచికలని మార్చు	Piece table (§5)	Immutable structures, git, COW filesystems
మార్పు ఒక వస్తువు	`InsertCommand` (§11)	Undo, transactions, job queues, event sourcing
వెనక్కి తిప్పే సమాచారం అప్పుడే పట్టుకో	`removed` (§11)	DB rollback, compensating transactions
చరిత్ర ఒక రేఖ, కొమ్మ కాదు	`#redo.length = 0` (§10)	Git, form state, wizard flows
సమయాన్ని తీసుకో, అడగొద్దు	`now` parameter (§12)	Deep Dive 04 §10 · testable time
Fuzz, tests కాదు	36,000 + 30,000 ops (§5, §13)	Parsers, serialisers, ఏ invertible operation ఐనా

ఒక చివరి ఆలోచన – ఈ PROBLEM LO నిజమైన కష్టం ఎక్కడ ఉంది

ఈ doc lo ఐదు సాంకేతిక ఆలోచనలు ఉన్నాయి – piece table, coalescing, commands, redo invalidation, fuzz testing. వాటిలో **ఒక్కటీ కష్టమైనది కాదు.**

కష్టమైనది వేరే: **ఈ సమస్యల్లో మూడు "తప్పు" లాగా కనిపించవు.** String version *పనిచేస్తుంది*. Snapshot undo *పనిచేస్తుంది*. Redo stack *పనిచేస్తుంది*. ప్రతి ఒక్కటీ tests pass అవుతుంది.

అవి విరిగేది – **పెద్ద document lo**, **ఒక గంట తర్వాత**, **ఒక నిర్దిష్ట వరుస చర్యల తర్వాత**. అంటే: **నిజమైన వాడకంలో, నిజమైన user దగ్గర.**

అందుకే ఈ doc lo సగం పని కొలవడం మరియు fuzz చేయడం. **సరిగ్గా అనిపించడం మరియు సరిగ్గా ఉండటం ఒకటి కాదు** – మరియు తేడాని బయటపెట్టే ఏకైక మార్గం, దాన్ని *వెతకడం*.

ఇక్కడి నుంచి ఎక్కడికి

ఈ series lo ఇప్పటివరకు: **01 Parking Lot** · **02 Cache** · **03 Rate Limiter** · **04 BookMyShow** · **05 Splitwise** · **06 Elevator** · **07 Pub-Sub** · **08 HashMap** · **09 Chess** · **10 Meeting Scheduler** · **11 Food Delivery** · **12 File System** · **13 Leaderboard** · **14 Text Editor**.

వేగవంతమైన revision కోసం – `LLD_Design_Problems_Telugu.pdf`.

Text Editor – అడుగు అడుగునా · ఈ doc lo ఉన్న ప్రతి output, ప్రతి సంఖ్య నిజంగా `node` lo run చేసి తీసినదే ✅